

BCC2001



Diego Bertolini

diegobertolini@gmail.com

Introdução a Ling. C

- **Aula Anterior:**

- Introdução

- **Aula de Hoje:**

- Introdução à Programação Estruturada.
- Introdução à linguagem C.
- Introdução à edição e compilação, compilação por linha de comando e uso de IDE.

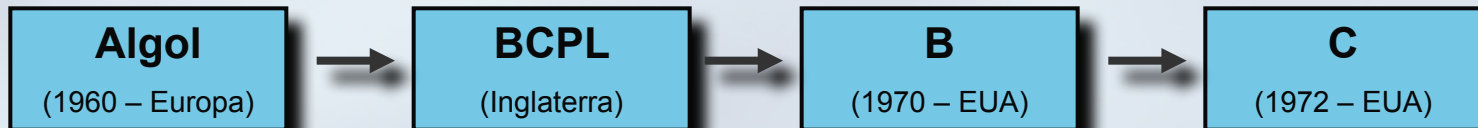
Objetivo

- **Apresentar uma introdução sobre programação estrutura na linguagem C e sobre as formas de edição e compilação de programas.**

Introdução a Linguagem C

- Linguagem C:

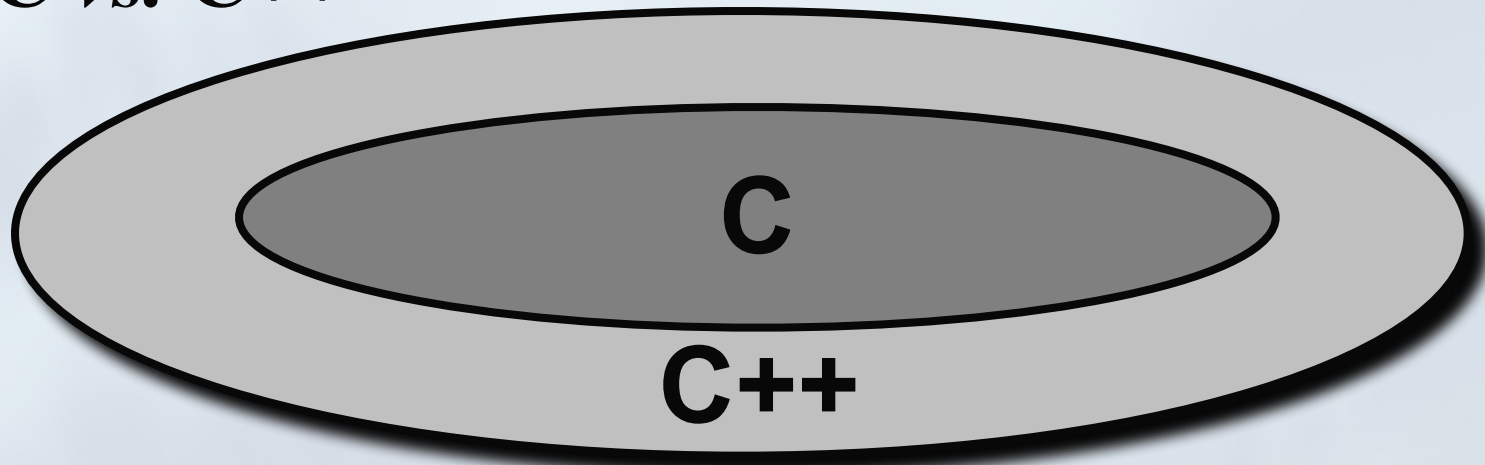
- Desenvolvida por Dennis Ritchie nos laboratórios AT&T Bell (EUA) no início dos anos 70;



- C é uma das linguagens de maior aceitação:
 - Portabilidade (Compiladores disponíveis para PC's, Mainframes, etc.)
 - Muitos compiladores C produzem código bastante otimizado;
 - Reúne tanto características de **alto nível** quanto de **baixo nível** (muitas vezes chamada de nível médio);
- Em 1983 foi criado o comitê ANSI-C;
- 1941-2011 - Turing 1983

Introdução a Linguagem C

- **C vs. C++**



- Linguagem C: linguagem de programação imperativa estruturada;
- Linguagem C++:
 - Linguagem Orientada a Objetos;
 - Formada pela linguagem C adicionada de um conjunto de instruções voltados à OO;
- Quem sabe programar em C conhece a maior parte de C++;

Introdução a Linguagem C

- Turbo C vs. Turbo Pascal
 - Termos básicos similares;
 - Algumas funções semelhantes;
 - C tem maior controle direto sobre o computador;
 - O C possui fraca verificação de erros;
 - A linguagem C é “*case sensitive*”;

“C é a linguagem de um programador”
Herbert Schildt

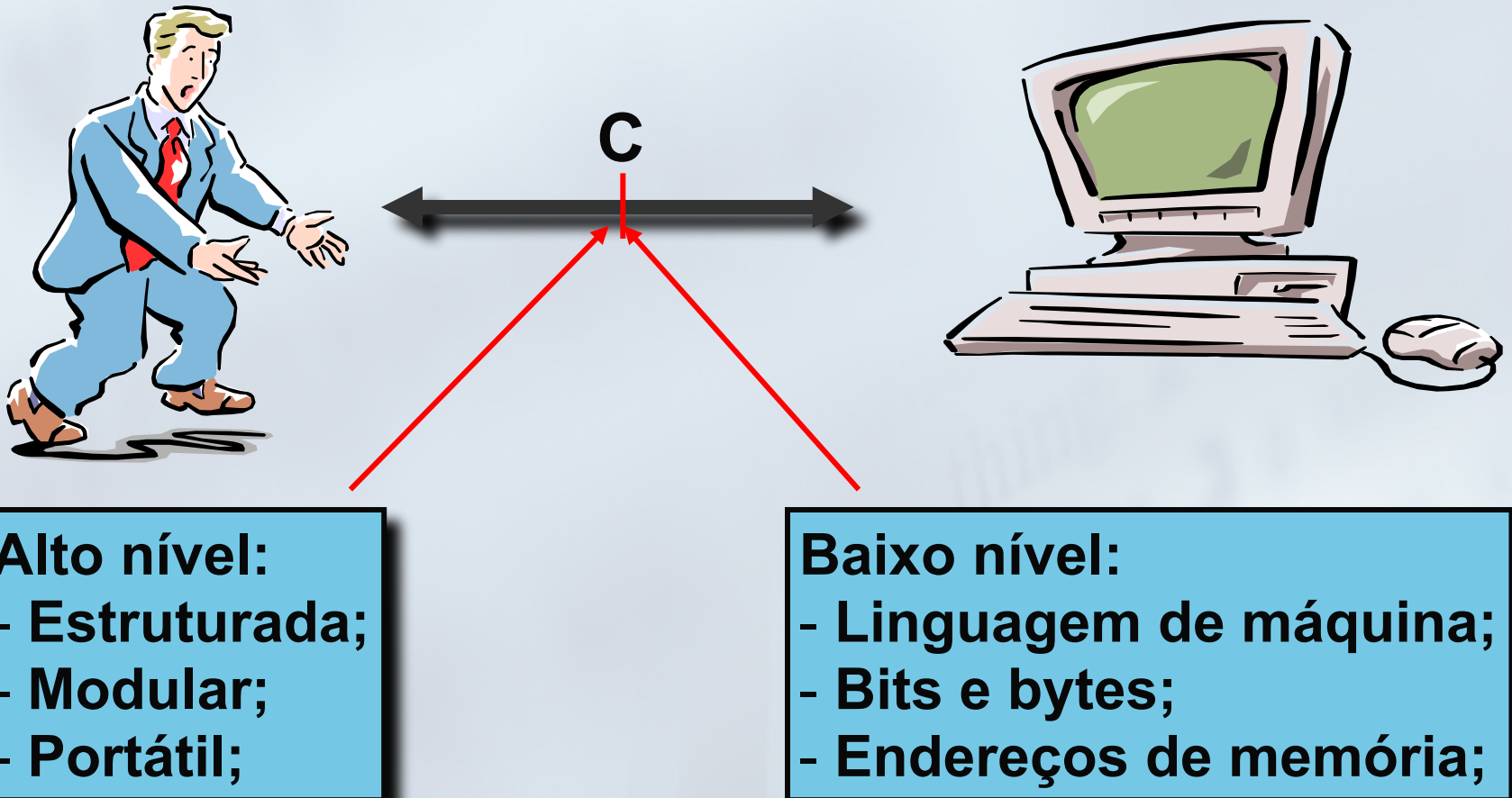
Introdução a Linguagem C

- A linguagem C é utilizada na criação de diversos tipos de aplicação, tais como:
 - Sistemas Operacionais (UNIX);
 - Processadores de Texto;
 - Planilhas Eletrônicas;
 - Sistemas Gerenciadores de BD;
 - Processadores Gráficos;
 - Problemas de Engenharia;
 - Compiladores;

“C é uma linguagem versátil”

Introdução a Linguagem C

- C: linguagem de nível médio;



Linguagens de programação

- Cada tipo de problema requer um algoritmo.
- Existem várias formas de implementar um algoritmo.
- Paradigmas para programação:
 - Estruturada
 - Orientada a objeto
 - Funcional

Linguagens de montagem (assembly)

- Linguagens de baixo nível, intimamente relacionadas à arquitetura do computador.
- Melhor do que código de máquina (ou cabos) mas pouca abstração.

Bubblesort
(assembly para PC)



```
DO_LOOP_TOP:
    MOV     SI,0
    MOV     DI,0
    INC     DI
    MOV     DH,0
    DEC     DL
FOR_LOOP:
    MOV     CH,0
    MOV     CL,DL
FOR_LOOP_TOP:
    JCXZ    FOR_LOOP_DONE
    MOV     AL,[BX+SI]
    CMP     AL,[BX+DI]
    JLE     NEXTI
    MOV     AH,[BX+DI]
    MOV     [BX+DI],AL
    MOV     [BX+SI],AH
    INC     DH
NEXTI:
    INC     SI
    INC     DI
    DEC     CX
    JMP     FOR_LOOP_TOP
FOR_LOOP_DONE:
    CMP     DH,0
    JG      DO_LOOP_TOP
    RET
```

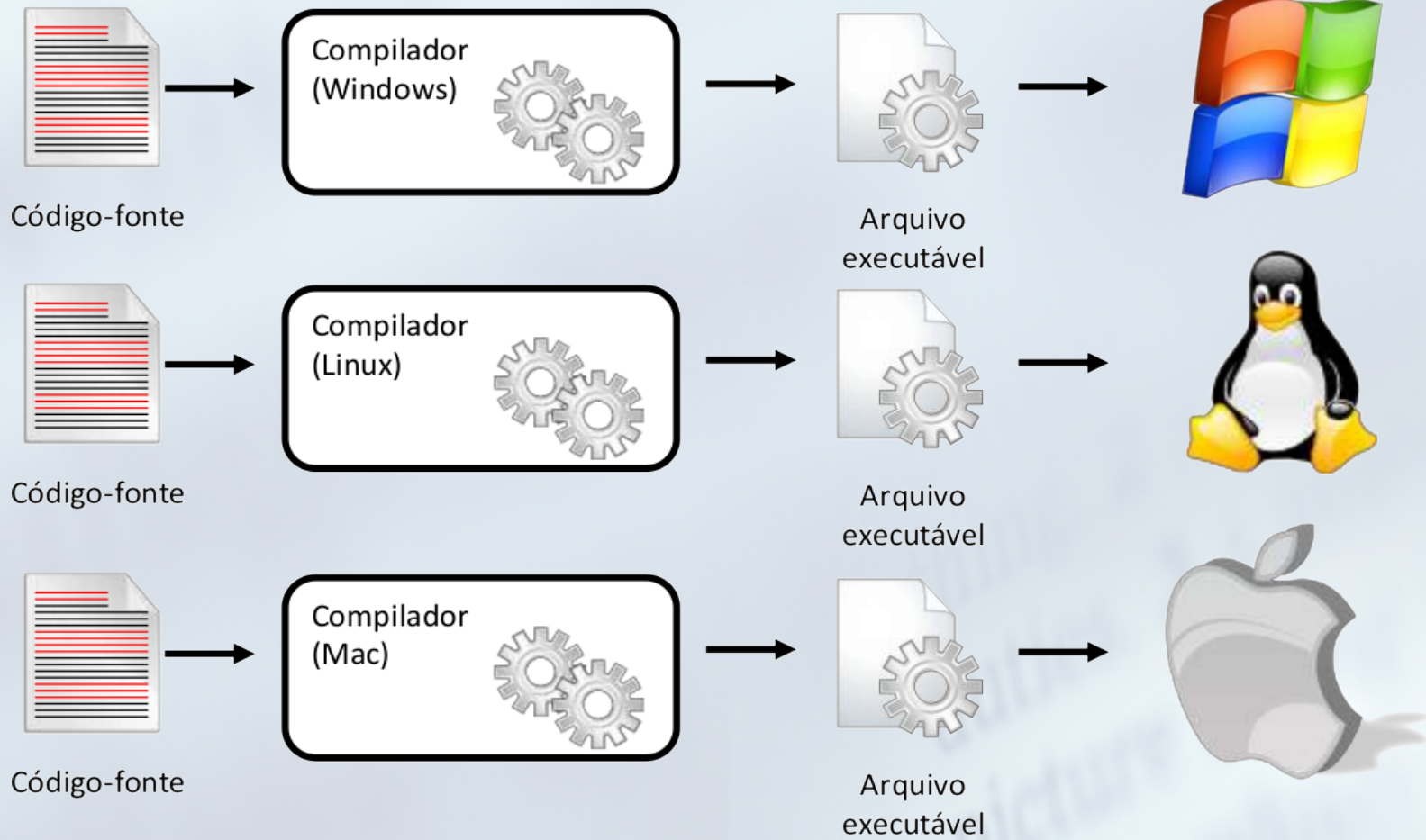
Compilador

- Transforma os programas que nós escreveremos em programas executáveis pelo computador.
- Transforma códigos de uma linguagem A para uma linguagem B;

```
#include <stdio.h>

int main() {
    printf("Hello world\n");
    return 0;
}
```

Compiladores



Introdução a Linguagem C

- Primeiros passos na Linguagem C:
 - *C é case sensitive:*
 - Pode-se declara quatro variáveis diferentes com os nomes **Soma**, **soma**, **SOMA** e **SoMa**, embora não seja recomendável na maioria das vezes;
 - Todas as palavras reservadas da linguagem C devem ser descritas em letra minúscula, assim temos:
 - **for** : palavra reservada que rotula a estrutura de repetição “para”;
 - **if** : palavra reservada que rotula a estrutura de seleção “se”;
 - **For** : poderia ser um nome de variável;
 - **If** : poderia ser um nome de variável;

Introdução a Linguagem C

- Estrutura básica de um programa em C:
 - Todo programa em C possui no mínimo uma função;
 - A função *main()* é obrigatória e é a primeira a ser executada. É a função principal, o ponto de partida do programa que será criado;
 - Por isso, a noção de função e de programa freqüentemente se confundem quando se programa em linguagem C;

Introdução a Linguagem C

- Forma geral de um programa em C:

inclusão de arquivos de cabeçalho

declaração de variáveis globais

main () → declaração da função principal (obrigatória). Os parênteses indicam que trata-se de uma função

{ → toda função começa com o símbolo “{”, equivale ao “início” (begin) de um bloco de instruções

declaração de variáveis locais

...

comandos;

...

} → toda função termina com o símbolo “}”, equivale ao “fim” (end) de um bloco de instruções

As instruções descritas no programa terminam sempre com “;”, como na linguagem Pascal.

Introdução a Linguagem C

- O menor programa possível em C seria:

```
main ()
```

```
{  
}
```

- É um programa vazio, que não contém instruções para executar;
- Formatação dos programas: é livre, mas convém manter a legibilidade (*ver slides livro);

```
main ()  
{  
}
```

```
main ()  
{}
```

Bons exemplos

```
main  
(  
)  
{  
}
```

```
main  
(  
{  
}
```

Maus exemplos

Introdução a Linguagem C

- Comentários: podem ser incluídos em qualquer parte do programa.
 - Se ocupar apenas uma linha, basta que a mesma seja iniciada com “//”;
 - Se ocupar mais de uma linha, o comentário deve iniciar com o símbolo “/*” e terminar com o símbolo “*/”;
- Palavras reservadas da Linguagem C:

auto	break	case	char	const	continue	default
do	double	else	enum	extern	float	for
goto	if	int	long	main	register	return
short	signed	sizeof	static	struct	switch	typedef
union	unsigned	void	volatile	while		

Programação Estruturada

- **Um programa é composto por blocos elementares de código que se interligam através de três mecanismos básicos:**
 - **Sequência:** implementa os passos para descrever qualquer programa.
 - **Seleção/Decisão:** possibilidade de selecionar/alterar o fluxo de execução do código baseado em ocorrências lógicas.
 - **Iteração/Repetição (Laços):** permite a execução repetitiva de blocos do programa.
- **Esses mecanismos se traduzem em estruturas nas linguagens de programação.**
- **Cada uma destas estruturas tem um ponto de início e um ponto de término de execução.**

Exemplo da aula passada

```
media.c ✕
1 #include <stdio.h>
2
3 int main(){
4     float n1, n2, m;
5
6     printf("Digite as duas notas:\n");
7     scanf("%f", &n1);
8     scanf("%f", &n2);
9
10    m = (n1 + n2) / 2;
11
12    printf("Média: %.2f\n", m);
13    if(m >= 7.0){
14        printf("Aprovado\n");
15    }
16    else{
17        printf("Reprovado\n");
18    }
19 }
```

A linguagem C

- Desenvolvida por **Dennis Ritchie** (**Laboratórios Bell, 1972**)
- Criada inicialmente para ser um "**ASSEMBLY portátil**".
- Rodava num computador PDP-11, usando o sistema operacional UNIX.
- **Posteriormente foi utilizada para reescrever o UNIX.**
- Somente em **1978** Brian W. **Kernighan** e Dennis **Ritchie** estabeleceram a **sintaxe** e a **semântica** da **linguagem C**.
- O desenvolvimento de C foi influenciado pela linguagem B (Ken Thompson), que teve suas origens na BCPL (Martins Richards).
- Tais linguagens surgiram da evolução da Algol CPL.

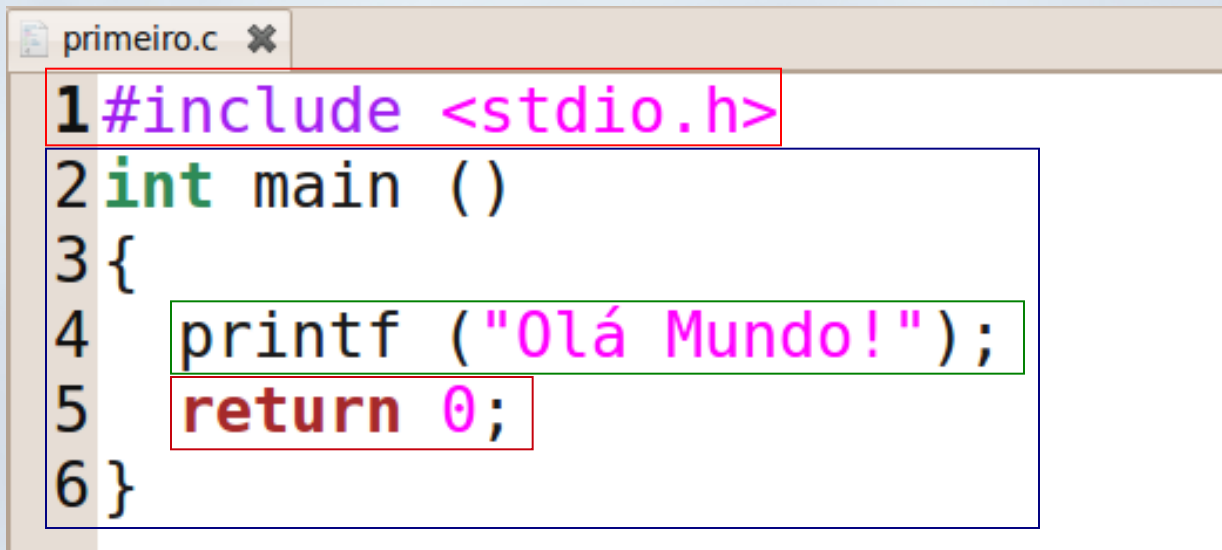
A linguagem C e sua família

- CPL (**C**ombined **P**rogramming **L**anguage)
- BCPL (Basic CPL)
- B (Ken Thompson, Bell Labs, 1970)
- **C (Dennis Ritchie, Bell Labs, 1972).**
- **C++ (Bjarne Stroustrup, Bell Labs, 1983)**

Estrutura de um programa em C

```
primeiro.c ✕  
1 #include <stdio.h>  
2 int main ()  
3 {  
4     printf ("Olá Mundo!");  
5     return 0;  
6 }
```

Estrutura de um programa em C



A screenshot of a text editor window titled "primeiro.c" showing a simple C program. The code is as follows:

```
1#include <stdio.h>
2int main ()
3{
4    printf ("Olá Mundo!");
5    return 0;
6}
```

The code is color-coded: line numbers 1-6 are in a light blue margin; `#include` is purple, `<stdio.h>` is magenta; `int` is green, `main` is black, and `()` is black; `{` is black; `printf` is black, `"Olá Mundo!"` is magenta, and `;` is black; `return` is red, `0` is black, and `;` is black; `}` is black.

Estrutura de um programa em C

Avisa ao compilador que as funções de entrada e saída de dados da biblioteca **stdio**

A função main é o corpo principal do programa.
int main() { ... }
Todo programa deve ter um main.
As chaves { e } marcam o **início** e o **fim** da função.

```
primeiro.c x
1 #include <stdio.h>
2 int main ()
3 {
4     printf ("Olá Mundo!");
5     return 0;
6 }
```

O programa usa a função **printf(...)** para imprimir a mensagem no console. Texto deve ser entre aspas: "Olá Mundo!"

Avisa o SO que o programa terminou sem problemas.

Partindo do que conhecemos: funções.

```
f(x) = x2 + 2x + 10  
g(x,y) = f(x) + y2  
h(x,y,z) = g(z,x) + f(y)  
h(1,4,5) = ?  
main() = h(1,4,5)
```

```
h(1,4,5) = g(5,1) + f(4)  
h(1,4,5) = [f(5) + 12] +  
           [42 + 2.4 + 10]  
h(1,4,5) = [[52+2.5+10] +  
           12] + [42 + 2.4 + 10]  
h(1,4,5) = [[25+10+10] + 1]  
           + [16 + 8 + 10]  
h(1,4,5) = [45 + 1] + [34]  
h(1,4,5) = 46 + 34  
H(1,4,5) = 80
```

Um código intermediário

```
f(x){  
  (x * x) + (2 * x) + 10;  
}
```

```
g(x,y){  
  f(x) + (y * y);  
}
```

```
h(x,y,z){  
  g(z,x) + f(y);  
}
```

```
main() {  
  "resultado: ", h(1,4,5);  
}
```

Código em C

```
#include <stdio.h>

float f(float x){
return (x * x) + (2 * x) + 10;
}

float g(float x,float y){
return f(x) + (y * y);
}

float h(float x,float y,float z){
return g(z,x) + f(y);
}

int main() {
printf("resultado: %f", h(1,4,5));

return 0;
}
```

Código em C

```
#include <stdio.h>
#include <math.h>

float f(float x){
    return pow(x,2) + (2 * x) + 10;
}

float g(float x,float y){
    return f(x) + pow(y,2);
}

float h(float x,float y,float z){
    return g(z,x) + f(y);
}

int main() {
    printf("resultado: %f", h(1,4,5));

    return 0;
}
```

Processo de edição e compilação

- Inicialmente iremos utilizar um editor de texto do sistema.
- O compilador será o gcc (já disponível no Linux)
- Instalaremos o Compilador na Aula Prática (quinta).

Variáveis e Constantes

- **Variáveis e constantes** são os **elementos básicos**.
- Uma variável corresponde a um **espaço reservado** na memória para **armazenar** um determinado tipo de dado.
- Variáveis devem receber **nomes significativos** que possam ser **referenciados** e modificados.
- É preciso **declarar** uma variável **antes de usá-la**.
- Declarações **especificam** de que **tipo** são as variáveis e as vezes um **valor inicial**.
- **Tipos podem ser por exemplo:** inteiros, reais, caracteres, etc.
- **Expressões** combinam **variáveis e constantes** para calcular novos valores.

Nomes das variáveis

- **Algumas regras básicas para a nomeação de variáveis:**
 - Todo nome só pode conter letras e dígitos;
 - O caractere "_" é contado como uma letra;
 - Todo primeiro caractere deve ser sempre uma letra;
 - Letras maiúsculas e minúsculas são consideradas caracteres diferentes → **case sensitive**.
 - Palavras reservadas não podem ser usadas como nome de variáveis.
- **É uma boa prática escolher nomes que significam alguma coisa e indiquem a função da variável.**
 - Por exemplo: valor, soma, total, nome, raio.

Declarando variáveis

- Para serem usadas, as variáveis precisam ser declaradas de modo que o compilador possa reservar espaço na memória para o valor a ser armazenado.
- A forma geral de uma declaração é:
 - `tipo lista_de_variaveis;`
- **Exemplos:**
 - `int i;`
 - `int a, b, c;`
 - `unsigned int dia, mes, ano;`
 - `double salario;`

Tipos de dados

- **Os dados em C podem assumir cinco tipos básicos:**
- ***char*:** Armazena um caractere. Caracteres geralmente são armazenados em códigos (código ASCII) e ocupam **um byte**.
- ***int*:** O valor armazenado é um número inteiro.
- ***float*:** Número em ponto flutuante de precisão simples. São os conhecidos números reais.
- ***double*:** Número em ponto flutuante de precisão dupla.
- ***void*:** Este tipo serve para indicar que um resultado não tem um tipo definido. Tipo vazio.

Modificadores de tipos

- **Modificadores** são palavras que alteram o tamanho do conjunto de valores que o tipo pode representar.
- **Por exemplo:** quero armazenar um inteiro que sei que não tem sinal.
- **Modificadores:**
 - ***unsigned***: Pode ser aplicado aos tipos ***int*** e ***char*** e faz com o bit de sinal não seja usado, ou seja o tipo passa a ter um bit a mais.
 - ***long***: Pode ser aplicado aos tipos ***int*** e ***double*** aumentando o número de bytes reservado para armazenamento de dados.

Modificadores de tipos

Tipo	Bytes	Faixa Mínima
char	1	-127 a 127
unsigned char	1	0 a 255
signed char	1	-127 a 127
int	4	-2.147.483.648 a 2.147.483.647
unsigned int	4	0 a 4.294.967.295
signed int	4	-2.147.483.648 a 2.147.483.647
short int, short	2	-32.768 a 32.767
unsigned short int	2	0 a 65.535
signed short int	2	-32.768 a 32.767
long int, long	4	-2.147.483.648 a 2.147.483.647
signed long int	4	-2.147.483.648 a 2.147.483.647
unsigned long int	4	0 a 4.294.967.295
long long int long long	8	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807
signed long long int signed long long	8	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807
unsigned long long int unsigned long long	8	0 a 18.446.744.073.709.551.615
float	4	oito dígitos de precisão
double	8	16 dígitos de precisão
long double	12	16 dígitos de precisão

Atribuição de valores

- Após ser declarada, uma variável pode receber valores.
- O operador de atribuição é: =
- Indicando que o resultado da expressão à direita do operador será atribuído à variável.
- Atribuições durante a declaração:

```
int i = 0, j = 10;          char c = 'd';  
float raio = 2.54;          double precisao = 0.00001 L;
```

Atribuições pós declaração:

```
int i;  
float raio ;  
j = 10;  
raio = 2.54;
```

Constantes

- Constantes não podem ser alteradas como as variáveis.

Pre-processor
Substituição textual

```
1 #include <stdio.h>
2
3 #define PI 3.14159265
4 #define NEW_LINE '\n'
5
6 int main ()
7 {
8     printf("Valor de PI: %f \n", PI);
9     printf("Valor de PI: %.|f \n", PI);
10    printf("Valor de PI: %.8f \n", PI);
11
12    printf ("Olá Mundo!%c", NEW_LINE);
13    return 0;
14 }
```

Funções de entrada e saída

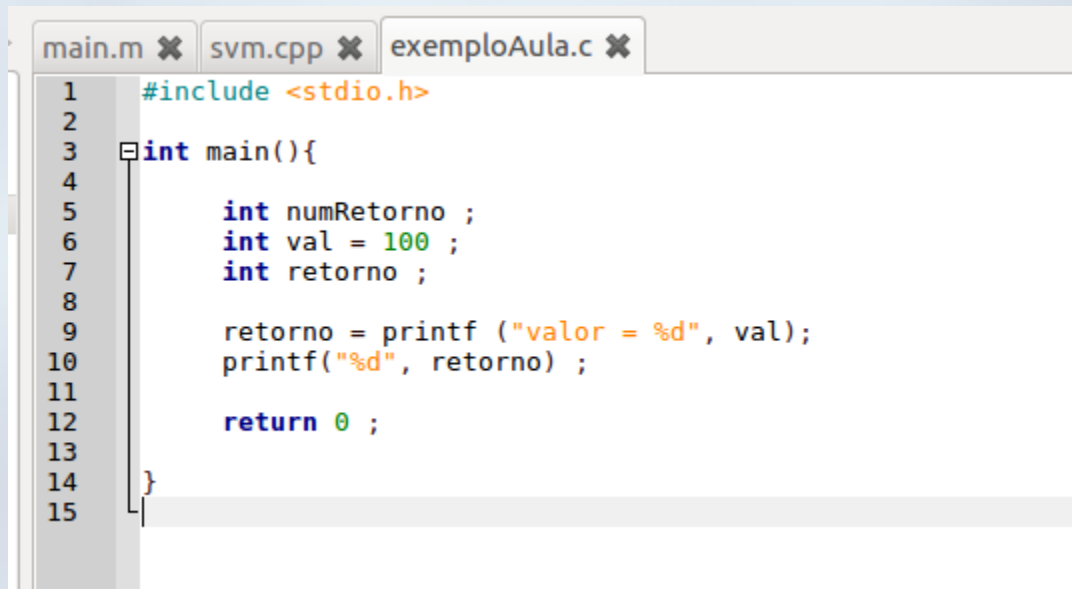
- A função `printf(...)` escreve dados na saída padrão, normalmente o terminal.
- O protótipo da função é:
- `int printf(controle, arg1, arg2, ...);`
- Onde os argumentos `arg1`, `arg2`, ... são impressos de acordo com o formato indicado pela cadeia de caracteres que compõe `controle`.
- O formato permite que os resultados possam ser apresentados de diversas maneiras
- A função retorna o número de caracteres impressos.
- No caso de um erro um valor negativo é retornado.

Caracteres para saída formatada

Código	Comentário
%c	Caracter simples
%d	Inteiro decimal com sinal
%i	Inteiro decimal com sinal
%E	Real em notação científica com E
%e	Real em notação científica com e
%f	Real em ponto flutuante
%G	%E ou %f, o que for mais curto
%g	%g ou %f, o que for mais curto
%o	Inteiro em base octal
%s	Cadeia Caracteres
%u	Inteiro decimal sem sinal
%x	Inteiro em base hexadecimal (letras minúsculas)
%X	Inteiro em base hexadecimal (letras maiúsculas)
%p	Endereço de memória
%%	Imprime o caractere %

Funções de entrada e saída

- Façam esse exemplo com a função printf.



```
main.m x svm.cpp x exemploAula.c x
1  #include <stdio.h>
2
3  int main(){
4
5      int numRetorno ;
6      int val = 100 ;
7      int retorno ;
8
9      retorno = printf ("valor = %d", val);
10     printf("%d", retorno) ;
11
12     return 0 ;
13
14 }
15
```

Funções de entrada e saída

- A função `scanf(...)` pode ser utilizada para entrada de dados a partir do teclado.
- Protótipo:
- `scanf(controle, &arg1, &arg2, ...);`
- Na `scanf` os argumentos que vem depois do controle são endereços das variáveis que irão receber os valores lidos e não como no `printf` que são as próprias variáveis. O `&` indica que estamos referenciando o endereço. Por exemplo:
- `scanf("%d %d", &a, &b);`
- Lê dois valores inteiros que serão armazenados na variável `a` e `b`, nos endereços indicados por `&a` e `&b` respectivamente.

Funções de entrada e saída

- Exemplo incluindo outros tipos de variáveis:

```
1#include <stdio.h>
2int main (void){
3
4    int i;
5    float f;
6    char c;
7
8    printf("Digite os valores: \n");
9    scanf("%d %f %c", &i, &f, &c);
10
11    printf("Valores digitados: \n");
12    printf("Inteiro: %d\n", i);
13    printf("Float: %f\n", f);
14    printf("Caracter: %c\n", c);
15
16    return 0;
17}
```

Exemplo

```
#define PI 3.1415

int main(void){
    int area, raio;

    printf("Digite o raio:\n");
    scanf("%f", &raio);

    area = PI * raio * raio;

    printf("Area: %f \n", area);
}
```

memória

Tela

Exemplo

```
#define PI 3.1415
```

```
int main(void){  
    int area, raio;  
  
    printf("Digite o raio:\n");  
    scanf("%f", &raio);  
  
    area = 3314455**raio**raio;  
  
    printf("Area: %f \n", area);  
}
```

memória

Tela

Exemplo

```
#define PI 3.1415
```

```
int main(void){
```

```
    int area, raio;
```

```
    printf("Digite o raio:\n");
```

```
    scanf("%f", &raio);
```

```
    area = 3.1415 * raio * raio;
```

```
    printf("Area: %f \n", area);
```

```
}
```

memória

Tela

Exemplo

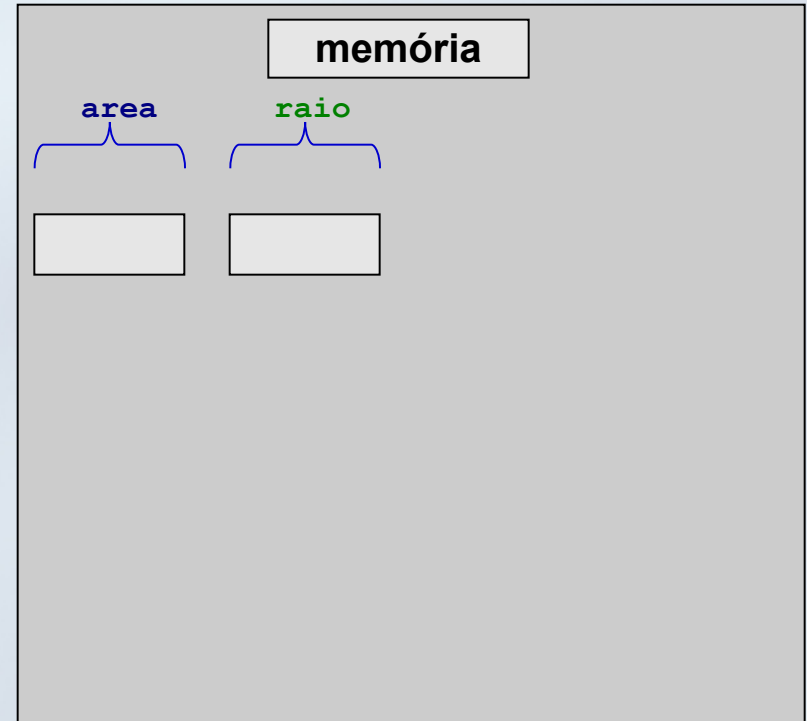
```
#define PI 3.1415

int main(void){
    int area, raio;

    printf("Digite o raio:\n");
    scanf("%f", &raio);

    area = 3.1415 * raio * raio;

    printf("Area: %f \n", area);
}
```



Exemplo

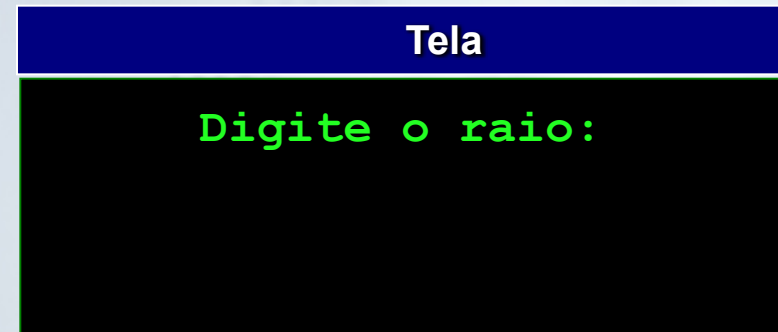
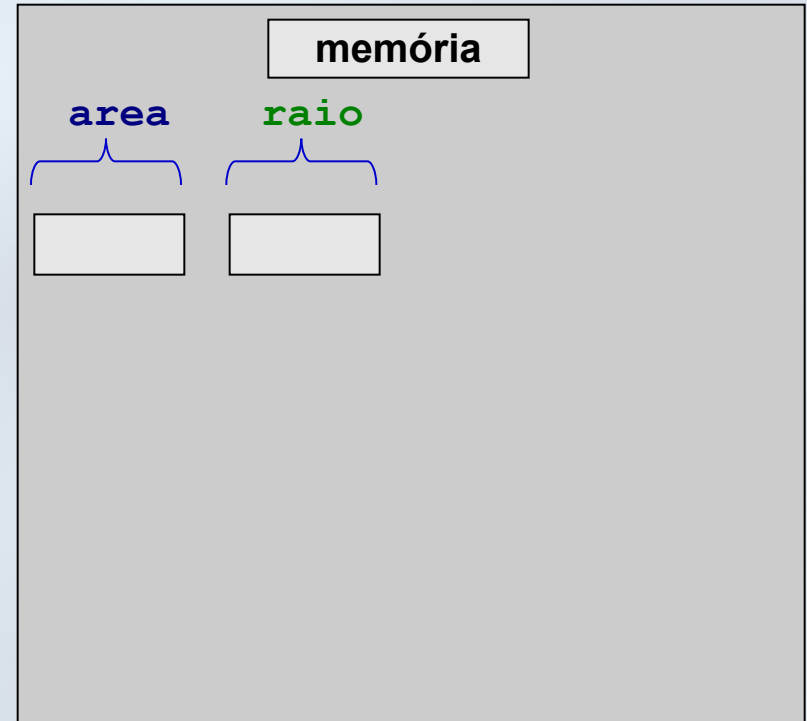
```
#define PI 3.1415

int main(void){
    int area, raio;

    printf("Digite o raio:\n");
    scanf("%f", &raio);

    area = 3.1415 * raio * raio;

    printf("Area: %f \n", area);
}
```



Exemplo

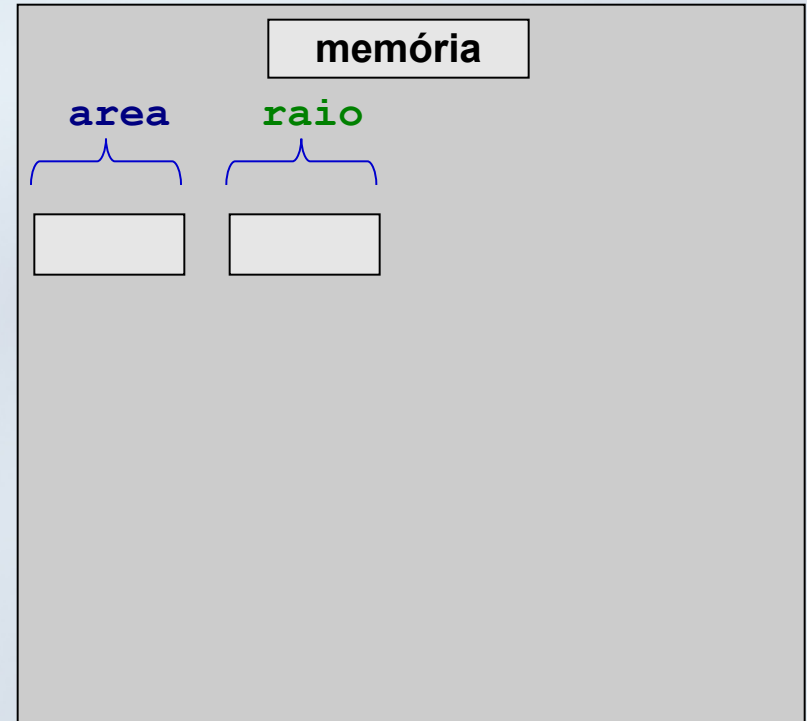
```
#define PI 3.1415

int main(void){
    int area, raio;

    printf("Digite o raio:\n");
    scanf("%f", &raio);

    area = 3.1415 * raio * raio;

    printf("Area: %f \n", area);
}
```



Exemplo

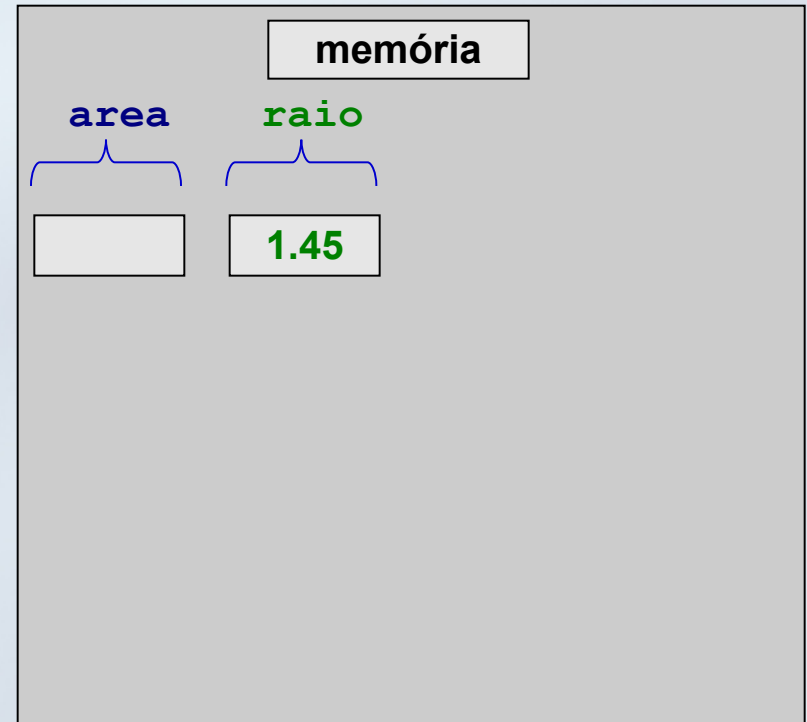
```
#define PI 3.1415

int main(void){
    int area, raio;

    printf("Digite o raio:\n");
    scanf("%f", &raio);

    area = 3.1415 * raio * raio;

    printf("Area: %f \n", area);
}
```



Exemplo

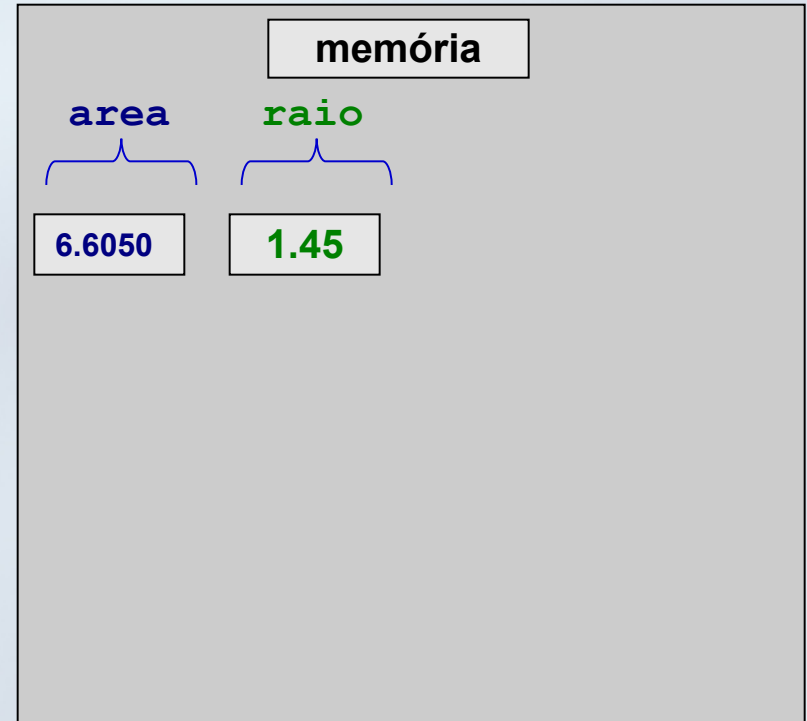
```
#define PI 3.1415

int main(void) {
    int area, raio;

    printf("Digite o raio:\n");
    scanf("%f", &raio);

    area = 3.1415 * raio * raio;

    printf("Area: %f \n", area);
}
```



Exemplo

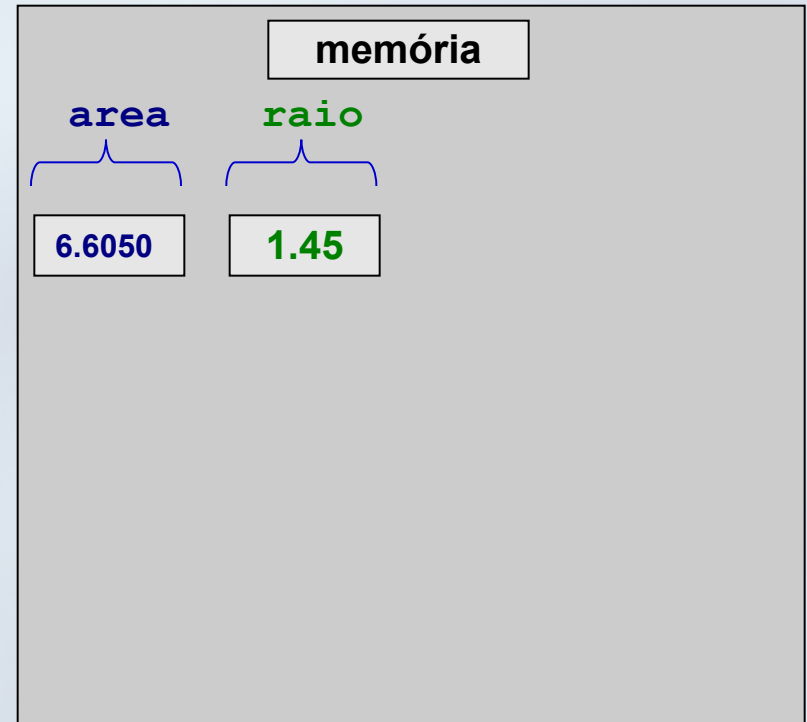
```
#define PI 3.1415

int main(void){
    float area, raio;

    printf("Digite o raio:\n");
    scanf("%f", &raio);

    area = 3.1415 * raio * raio;

    printf("Area: %f \n", area);
}
```



Tela

```
Digite o raio:
1.45
Area: 6.6050
```

Exercícios

- Indique os nomes de variáveis que são válidos. Justifique os nomes inválidos.
- tempo
- nota_final
- us\$
- char
- 2dias
- teste 1
- raio.do.circulo

Exercícios da lista

1. **Faça um algoritmo que imprima a seguinte mensagem: “Só se aprende algoritmo, fazendo algoritmo. É preciso fazer todos os exercícios para aprender.”**
2. **Leia um número e imprima seu sucessor e seu antecessor.**
3. **Faça um algoritmo que receba 3 números, calcule e mostre a multiplicação desses números.**
4. **Faça um programa que receba duas notas, calcule e mostre a média ponderada dessas notas, considerando peso 0.3 para a primeira nota e peso 0.7 para a segunda nota.**

Dúvidas, Críticas ou Sugestões

diegobertolini@gmail.com